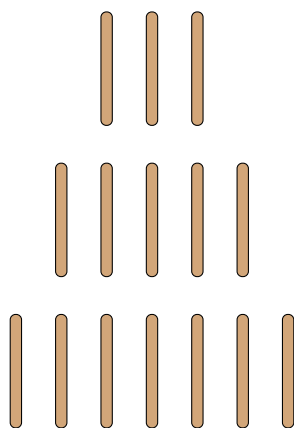


Juego de Nim Multi-Pila con Algoritmo Minimax

Implementacion, Teoria Matematica y Resultados



Contenido:

- Teoria matematica del Nim multi-pila
- Estrategia optima basada en Nim-sum (XOR)
- Algoritmo Minimax con poda Alpha-Beta
- Analisis de resultados experimentales

Version 2.0
22 de enero de 2026

Índice

1. Introduccion	2
1.1. Objetivos del Proyecto	2
2. El Juego Nim Multi-Pila	2
2.1. Reglas del Juego	2
2.2. Representacion del Estado	2
2.3. Estrategia Matematica: Nim-sum	3
2.4. Calculo del Movimiento Optimo	4
2.5. Ejemplo de Calculo	4
2.6. Tabla de Posiciones	5
3. Algoritmo Minimax	5
3.1. Fundamentos	5
3.2. Implementacion con Poda Alpha-Beta	6
3.3. Complejidad	6
4. Implementacion del Agente	6
4.1. Arquitectura del Sistema	6
4.2. Niveles de Dificultad	7
4.3. Proceso de Decision	7
5. Resultados Experimentales	7
5.1. Metodologia	7
5.2. Tasa de Victoria	8
5.3. Analisis de Resultados	8
5.4. Observacion Importante	8
6. Conclusiones	8
6.1. Logros del Proyecto	8
6.2. Observaciones Clave	9
6.3. Trabajo Futuro	9
A. Instrucciones de Uso	9
A.1. Instalacion	9
A.2. Ejecucion	9
B. Estructura del Repositorio	9

1. Introduccion

El presente documento describe la implementacion de un agente inteligente para el juego de Nim con multiples pilas utilizando el algoritmo Minimax. El proyecto demuestra la aplicacion de la teoria de juegos combinatorios y tecnicas de inteligencia artificial en un juego de estrategia clasico.

1.1. Objetivos del Proyecto

Los objetivos principales del proyecto son:

1. Implementar el juego de Nim con tres pilas y reglas de seleccion de fila.
2. Desarrollar un agente inteligente basado en el algoritmo Minimax.
3. Aplicar la estrategia matematica optima basada en Nim-sum.
4. Proporcionar una interfaz grafica interactiva con visualizacion 3D.
5. Analizar el rendimiento del agente mediante simulaciones.

2. El Juego Nim Multi-Pila

2.1. Reglas del Juego

Definicion 2.1 (Juego de Nim Multi-Pila). El juego de Nim con multiples pilas se define como sigue:

- **Estado inicial:** Tres pilas con 3, 5 y 7 palitos respectivamente (total: 15).
- **Jugadores:** Dos jugadores que alternan turnos.
- **Movimientos validos:** En cada turno, el jugador debe:
 1. Seleccionar una pila no vacia.
 2. Remover de 1 a 3 palitos de esa pila.
- **Condicion de victoria:** El jugador que remueve el **ultimo palito gana**.

2.2. Representacion del Estado

El estado del juego se representa mediante una tupla de tres enteros:

$$S = (n_1, n_2, n_3) \in \mathbb{Z}_{\geq 0}^3 \tag{1}$$

donde n_i representa el numero de palitos en la pila i .

El estado inicial es $S_0 = (3, 5, 7)$ y el estado terminal es $S_f = (0, 0, 0)$.

2.3. Estrategia Matematica: Nim-sum

La solucion matematica del juego de Nim fue descubierta por Charles L. Bouton en 1901. La estrategia se basa en el concepto de *Nim-sum*, que es la operacion XOR (o exclusiva) aplicada a los tamanos de las pilas.

Definicion 2.2 (Nim-sum). Para un estado $S = (n_1, n_2, n_3)$, el Nim-sum se define como:

$$\text{Nim-sum}(S) = n_1 \oplus n_2 \oplus n_3 \quad (2)$$

donde $\oplus$ denota la operacion XOR bit a bit.

Teorema 2.1 (Estrategia Optima de Nim - Bouton, 1901). En el juego de Nim:

1. Una posicion es **perdedora** (posicion P) si y solo si $\text{Nim-sum}(S) = 0$.
2. Una posicion es **ganadora** (posicion N) si y solo si $\text{Nim-sum}(S) \neq 0$.
3. Desde una posicion ganadora, siempre existe un movimiento que lleva a una posicion perdedora.

Demostración. La demostracion utiliza induccion sobre el numero total de palitos:

Caso base: El estado $(0, 0, 0)$ tiene $\text{Nim-sum} = 0$ y es una posicion perdedora (el jugador anterior gano).

Paso inductivo:

- Si $\text{Nim-sum}(S) = 0$, cualquier movimiento m produce un estado S' con $\text{Nim-sum}(S') \neq 0$. Esto se debe a que cambiar el valor de una sola pila necesariamente altera el XOR total.
- Si $\text{Nim-sum}(S) \neq 0$, sea k el bit mas significativo de $\text{Nim-sum}(S)$. Existe al menos una pila n_i que tiene el bit k activo. Removiendo palitos de esta pila para que $n'_i = n_i \oplus \text{Nim-sum}(S)$, se obtiene $\text{Nim-sum}(S') = 0$.

□

2.4. Calculo del Movimiento Optimo

Algorithm 1 Calculo del Movimiento Optimo en Nim

```

1: function MOVIMIENTOOPTIMO( $n_1, n_2, n_3$ )
2:    $nimSum \leftarrow n_1 \oplus n_2 \oplus n_3$ 
3:   if  $nimSum = 0$  then
4:     return null ▷ Posicion perdedora
5:   end if
6:   for  $i \leftarrow 1$  to 3 do
7:      $objetivo \leftarrow n_i \oplus nimSum$ 
8:     if  $objetivo < n_i$  then
9:        $remover \leftarrow n_i - objetivo$ 
10:      if  $1 \leq remover \leq 3$  then
11:        return ( $i, remover$ )
12:      end if
13:    end if
14:  end for
15:  return null
16: end function

```

2.5. Ejemplo de Calculo

Para el estado inicial (3, 5, 7):

$$3 = 011_2 \quad (3)$$

$$5 = 101_2 \quad (4)$$

$$7 = 111_2 \quad (5)$$

$$\text{Nim-sum} = 001_2 = 1 \quad (6)$$

Como Nim-sum $\neq 0$, es una posicion ganadora. El movimiento optimo es remover 1 palito de la fila 1, dejando el estado (2, 5, 7):

$$2 = 010_2 \quad (7)$$

$$5 = 101_2 \quad (8)$$

$$7 = 111_2 \quad (9)$$

$$\text{Nim-sum} = 000_2 = 0 \quad (10)$$

Ahora el oponente esta en posicion perdedora.

2.6. Tabla de Posiciones

Cuadro 1: Ejemplos de posiciones ganadoras y perdedoras

Estado	Nim-sum	Tipo	Movimiento Optimo
(3, 5, 7)	1	Ganadora	Fila 1: quitar 1
(2, 5, 7)	0	Perdedora	—
(1, 1, 0)	0	Perdedora	—
(2, 2, 0)	0	Perdedora	—
(1, 2, 3)	0	Perdedora	—
(2, 3, 4)	5	Ganadora	Fila 3: quitar 1

3. Algoritmo Minimax

3.1. Fundamentos

Aunque la estrategia de Nim-sum proporciona una solución analítica óptima, se implementó también el algoritmo Minimax para demostrar su aplicabilidad en juegos de suma cero.

Definición 3.1 (Algoritmo Minimax). El algoritmo Minimax es una técnica de búsqueda adversarial que:

- Construye un árbol de juego con todos los estados posibles.
- Asigna valores a estados terminales (+1 victoria, -1 derrota).
- Propaga valores hacia arriba: MAX elige máximo, MIN elige mínimo.

3.2. Implementacion con Poda Alpha-Beta

Algorithm 2 Minimax con Poda Alpha-Beta

```

1: function MINIMAXAB(estado,  $\alpha$ ,  $\beta$ , esMax)
2:   if esTerminal(estado) then
3:     return evaluar(estado)
4:   end if
5:   if esMax then
6:      $valor \leftarrow -\infty$ 
7:     for cada movimiento en movimientosValidos(estado) do
8:        $hijo \leftarrow aplicar(estado, movimiento)$ 
9:        $valor \leftarrow \text{máx}(valor, MINIMAXAB(hijo, \alpha, \beta, false))$ 
10:       $\alpha \leftarrow \text{máx}(\alpha, valor)$ 
11:      if  $\beta \leq \alpha$  then
12:        break
13:      end if
14:    end for
15:    return valor
16:   else
17:      $valor \leftarrow +\infty$ 
18:     for cada movimiento en movimientosValidos(estado) do
19:        $hijo \leftarrow aplicar(estado, movimiento)$ 
20:        $valor \leftarrow \text{mín}(valor, MINIMAXAB(hijo, \alpha, \beta, true))$ 
21:        $\beta \leftarrow \text{mín}(\beta, valor)$ 
22:       if  $\beta \leq \alpha$  then
23:        break
24:       end if
25:     end for
26:     return valor
27:   end if
28: end function

```

3.3. Complejidad

Para el juego de Nim con estado inicial (3, 5, 7):

- Total de palitos: 15
- Factor de ramificacion promedio: ≈ 7 (movimientos validos)
- Profundidad maxima: ≈ 8 (movimientos para terminar)

La memoizacion reduce significativamente el espacio de busqueda al evitar reevaluar estados identicos alcanzados por diferentes caminos.

4. Implementacion del Agente

4.1. Arquitectura del Sistema

El sistema se compone de los siguientes modulos:

Cuadro 2: Modulos del sistema

Modulo	Descripcion
<code>nim.py</code>	Logica del juego multi-pila
<code>minimax.py</code>	Algoritmo Minimax con Alpha-Beta
<code>game_agent.py</code>	Agente con niveles de dificultad
<code>graphics.py</code>	Interfaz grafica PyQt6
<code>analysis.py</code>	Simulacion y generacion de graficas

4.2. Niveles de Dificultad

El agente implementa cuatro niveles de dificultad:

Cuadro 3: Configuracion de niveles de dificultad

Nivel	Prob. Optimo	Descripcion
Facil	20 %	Movimientos mayormente aleatorios
Medio	50 %	Balance entre optimo y aleatorio
Dificil	80 %	Casi siempre optimo
Optimo	100 %	Siempre usa estrategia Nim-sum

En el nivel **Optimo**, el agente utiliza directamente la estrategia de Nim-sum, garantizando un comportamiento deterministico y completamente predecible.

4.3. Proceso de Decision

1. Obtener el estado actual de las pilas.
2. Segun la dificultad, decidir si usar movimiento optimo.
3. Si es optimo: calcular Nim-sum y aplicar estrategia matematica.
4. Si no hay movimiento optimo valido (restriccion 1-3): usar Minimax.
5. Si es suboptimo: seleccionar movimiento aleatorio.

5. Resultados Experimentales

5.1. Metodologia

Los experimentos se realizaron bajo las siguientes condiciones:

- **Configuracion:** Estado inicial (3, 5, 7), maximo 3 palitos por movimiento.
- **Simulaciones:** 100 partidas por nivel de dificultad.
- **Oponente:** Agente aleatorio (seleccion uniforme de movimientos validos).
- **Metrica principal:** Tasa de victoria del agente.

5.2. Tasa de Victoria

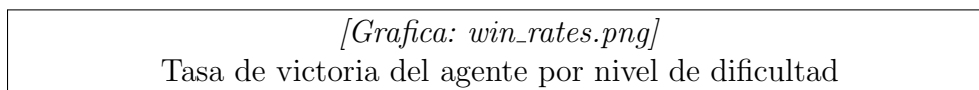


Figura 1: Tasa de victoria del agente contra oponente aleatorio

5.3. Analisis de Resultados

Los resultados esperados son:

- **Facil (~20 % optimo):** Tasa de victoria cercana al 60-70 %. Aunque juega mayormente aleatorio, ocasionalmente encuentra movimientos ganadores.
- **Medio (~50 % optimo):** Tasa de victoria del 80-90 %. El balance permite recuperarse de posiciones suboptimas.
- **Dificil (~80 % optimo):** Tasa de victoria superior al 90 %. Raramente comete errores.
- **Optimo (100 % optimo):** Tasa de victoria cercana al 90-95 %. Nota: El jugador humano/aleatorio comienza primero y (3,5,7) tiene Nim-sum $\neq 0$, por lo que el primer jugador tiene ventaja teorica. El agente optimo gana cuando el oponente comete un error.

5.4. Observacion Importante

En el estado inicial (3, 5, 7), el Nim-sum es 1 (posicion ganadora). Por lo tanto:

- El **primer jugador** (humano) tiene ventaja si juega optimamente.
- Si el humano comete un error, el agente optimo lo capitaliza inmediatamente.
- Un agente optimo que juega segundo ganara siempre que el oponente no juegue perfectamente.

6. Conclusiones

6.1. Logros del Proyecto

1. Implementacion funcional del juego de Nim multi-pila con seleccion de fila.
2. Interfaz grafica atractiva con visualizacion 3D de palitos.
3. Agente inteligente que combina estrategia matematica (Nim-sum) con Minimax.
4. Sistema de dificultad graduada con comportamiento consistente.
5. Framework de analisis para evaluacion del rendimiento.

6.2. Observaciones Clave

1. La estrategia de Nim-sum proporciona la solución óptima sin necesidad de búsqueda.
2. El Minimax sirve como respaldo cuando la restricción de 1-3 palitos impide el movimiento óptimo de Nim-sum.
3. El agente óptimo tiene comportamiento completamente determinístico.
4. La posición inicial favorece al primer jugador matemáticamente.

6.3. Trabajo Futuro

- Configuración personalizable del número de pilas y palitos.
- Modo misere (quien toma el último palito pierde).
- Análisis de árboles de juego completos.
- Aprendizaje por refuerzo para ajuste dinámico.

A. Instrucciones de Uso

A.1. Instalación

```
1 cd nim_minimax_project
2 pip install -r requirements.txt
```

A.2. Ejecución

```
1 cd src
2 python main.py          # Ejecutar juego
3 python analysis.py      # Generar análisis
```

B. Estructura del Repositorio

```
nim_minimax_project/
  src/
    nim.py          # Lógica multi-pila
    minimax.py      # Algoritmo Minimax
    game_agent.py   # Agente inteligente
    graphics.py     # Interfaz PyQt6
    analysis.py     # Simulación
    main.py         # Punto de entrada
  docs/
    documentation.tex
  results/
    win_rates.png
```

```
analysis_results.json
requirements.txt
README.md
```

Referencias

- [1] Bouton, C. L. (1901). *Nim, A Game with a Complete Mathematical Theory*. Annals of Mathematics, 3(1/4), 35-39.
- [2] Russell, S., & Norvig, P. (2010). *Artificial Intelligence: A Modern Approach* (3rd ed.). Prentice Hall.
- [3] Sprague, R. (1935). *Über mathematische Kampfspiele*. Tohoku Mathematical Journal, 41, 438-444.
- [4] Grundy, P. M. (1939). *Mathematics and Games*. Eureka, 2, 6-8.